PROJETO TMBA - Google Play Store App

Projeto desenvolvido por:

- Débora Jesus, nº2250064
- João Vasques, nº2250055
- Rui Filipe Parada, nº2211025

LLM: RoBERTa

1. Imports and Data Upload

```
# Instalação de bibliotecas adicionais
!pip install -q transformers datasets scikit-learn
!pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.2
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from dataset
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from data
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from hu
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggin
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requ
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requ
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->da
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohtt
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aioht
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aio
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohtt
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aioht
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateuti
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->hug
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from type
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->hug
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from ty
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from r
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-
```

```
# Imports principais
from google.colab import drive
from google.colab import files
```

```
import pandas as pd
import numpy as np
import io
import torch

# Pré-processamento e métricas
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    cohen_kappa_score,
    confusion_matrix
)

# Hugging Face - tokenização, modelo e treino
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

# Conversão para dataset compatível com Hugging Face
from datasets import Dataset
```

```
# Upload do dataset (em csv)

uploaded = files.upload()

review_df = pd.read_csv(io.BytesIO(uploaded['google_play_store_apps.csv']), sep=',')
```

```
Escolher Ficheiros google_play_store_apps.csv
google_play_store_apps.csv(text/csv) - 6436021 bytes, last modified: 16/03/2026 - 100% done
Saving google_play_store_apps.csv to google_play_store_apps.csv
```

```
review_df.head()
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.00	0.533333
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.25	0.288462
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.40	0.875000
3	10 Best Foods for	Best idea us	Positive	1.00	0.300000

Passos seguintes: Gerar código com review_df | New interactive sheet

2. Pré-processamento

```
# Remover reviews com sentimento neutro
review_df = review_df[review_df['Sentiment'] != 'Neutral']

# Visualizar as primeiras linhas
display(review_df.head(10))
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.000000	0.533333
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.250000	0.288462
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.400000	0.875000
3	10 Best Foods for You	Best idea us	Positive	1.000000	0.300000
4	10 Best Foods for You	Best way	Positive	1.000000	0.300000
5	10 Best Foods for You	Amazing	Positive	0.600000	0.900000
8	10 Best Foods for You	good you.	Positive	0.700000	0.600000
	10 Best	Useful information The amount			

3. Fine Tuning

3.1. Codificação dos Rótulos

```
# Codificar sentimentos em valores numéricos
le = LabelEncoder()
review_df["label"] = le.fit_transform(review_df["Sentiment"])

# Mostrar primeiras linhas
display(review_df.head(10))
```

	App	Translated_Review	Sentiment	Sentiment_Polarity	Sentiment_Subjectivity	label
0	10 Best Foods for You	I like eat delicious food. That's I'm cooking ...	Positive	1.000000	0.533333	1
1	10 Best Foods for You	This help eating healthy exercise regular basis	Positive	0.250000	0.288462	1
2	10 Best Foods for You	Works great especially going grocery store	Positive	0.400000	0.875000	1
3	10 Best Foods for You	Best idea us	Positive	1.000000	0.300000	1
4	10 Best Foods for You	Best way	Positive	1.000000	0.300000	1
5	10 Best Foods for You	Amazing	Positive	0.600000	0.900000	1
8	10 Best Foods for You	good you.	Positive	0.700000	0.600000	1
9	10 Best Foods for You	Useful information The amount spelling errors ...	Positive	0.200000	0.100000	1
10	10 Best Foods for You	Thank you! Great app!! Add arthritis, eyes, im...	Positive	0.750000	0.875000	1
11	10 Best Foods for You	Greatest ever Completely awesome maintain heal...	Positive	0.992188	0.866667	1

3.2. Separação dos Dados em Treino e Teste

```
# Separar dados de treino (80%) e teste (20%)
train_df = review_df.sample(frac=0.8, random_state=42)
test_df = review_df.drop(train_df.index)
```

```
# Mostrar dimensões dos conjuntos
print("train_df:", train_df.shape)
print("→", train_df.shape[0], "linhas (documentos de treino) e", train_df.shape[1], "colunas (atributo

print("\ntest_df:", test_df.shape)
print("→", test_df.shape[0], "linhas (documentos de teste) e", test_df.shape[1], "colunas (atributos s
```

```
train_df: (25815, 6)
→ 25815 linhas (documentos de treino) e 6 colunas (atributos seleccionadas)

test_df: (6454, 6)
→ 6454 linhas (documentos de teste) e 6 colunas (atributos seleccionadas)
```

3.3. Conversão para Dataset Hugging Face

```
# Converter DataFrames para Dataset Hugging Face
train_dataset = Dataset.from_pandas(train_df[["Translated_Review", "label"]])
test_dataset = Dataset.from_pandas(test_df[["Translated_Review", "label"]])
```

```
# Mostrar o conteúdo
display(train_dataset)
```

```
display(test_dataset)
```

```
Dataset({
    features: ['Translated_Review', 'label', '__index_level_0__'],
    num_rows: 25815
})
Dataset({
    features: ['Translated_Review', 'label', '__index_level_0__'],
    num_rows: 6454
})
```

3.4. Tokenização dos Conjuntos de Treino e Teste

```
# Carregar tokenizador do modelo Twitter-RoBERTa
tokenizer = AutoTokenizer.from_pretrained("cardiffnlp/twitter-roberta-base-sentiment")

# Tokenizar treino
train_encodings = tokenizer(
    list(train_dataset["Translated_Review"]),
    truncation=True,
    padding=True,
    max_length=512
)

# Tokenizar teste
test_encodings = tokenizer(
    list(test_dataset["Translated_Review"]),
    truncation=True,
    padding=True,
    max_length=512
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Pl
config.json: 100%  747/747 [00:00<00:00, 80.1kB/s]
vocab.json:  899k/? [00:00<00:00, 14.0MB/s]
merges.txt:  456k/? [00:00<00:00, 21.5MB/s]
special_tokens_map.json: 100%  150/150 [00:00<00:00, 15.2kB/s]
```

```
# Adicionar tokens ao treino
train_dataset = train_dataset.add_column("input_ids", train_encodings["input_ids"])
train_dataset = train_dataset.add_column("attention_mask", train_encodings["attention_mask"])

# Adicionar tokens ao teste
test_dataset = test_dataset.add_column("input_ids", test_encodings["input_ids"])
test_dataset = test_dataset.add_column("attention_mask", test_encodings["attention_mask"])
```

3.5. Carregamento do Modelo e Definição dos Parâmetros de Treino

```
# Converter datasets para formato PyTorch
train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

# Carregar modelo de classificação de sentimento
model = AutoModelForSequenceClassification.from_pretrained(
    "cardiffnlp/twitter-roberta-base-sentiment"
)
```

```
# Definir parâmetros de treino
training_args = TrainingArguments(
    output_dir="./output",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=2,
    report_to="none"
)
```

```
pytorch_model.bin: 100%                                                 499M/499M [00:05<00:00, 118MB/s]

Loading weights: 100%        201/201 [00:00<00:00, 719.19it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]

model.safetensors:   0%                                                 0.00/499M [00:00<?, ?B/s]

RobertaForSequenceClassification LOAD REPORT from: cardiffnlp/twitter-roberta-base-sentiment
Key                             | Status     |  |
--------------------------------+------------+--+-
roberta.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect ide
```

3.6. Definição da Métrica de Avaliação

```
def calcular_metricas(eval_pred):

    logits, labels = eval_pred

    preds = np.argmax(logits, axis=1)

    acc = accuracy_score(labels, preds)
    prec = precision_score(labels, preds, average='weighted')
    rec = recall_score(labels, preds, average='weighted')
    f1 = f1_score(labels, preds, average='weighted')
    kappa = cohen_kappa_score(labels, preds)

    cm = confusion_matrix(labels, preds)

    tn, fp, fn, tp = cm.ravel()

    taxa_erro = (fp + fn) / (tp + tn + fp + fn)

    return {
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
        "kappa": kappa,
        "taxa_erro": taxa_erro
    }
```

3.7. Treino e Avaliação do Modelo

```
# Configurar treino
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=calcular_metricas
)

# Treinar modelo
trainer.train()
```



[12908/12908 1:59:40, Epoch 2/2]

Step	Training Loss
500	0.507782
1000	0.451089
1500	0.423067
2000	0.366064
2500	0.412666
3000	0.420817
3500	0.359279
4000	0.388066
4500	0.351169
5000	0.301010
5500	0.297496
12000	0.160100
12500	0.151145

```
# Avaliar modelo
resultados = trainer.evaluate()

# Converter resultados para DataFrame
df_resultados = pd.DataFrame([{
    "Model": "RoBERTa",
    "Acc": resultados["eval_accuracy"],
    "Prec": resultados["eval_precision"],
    "Rec": resultados["eval_recall"],
    "F1": resultados["eval_f1"],
    "Kappa": resultados["eval_kappa"],
    "Taxa_Erro": resultados["eval_taxa_erro"]
}])

# Mostrar tabela
print(df_resultados)

# Exportar CSV
df_resultados.to_csv("RoBERTa_results.csv", index=False)
```

[1614/1614 01:59]

```
    Model       Acc      Prec       Rec        F1    Kappa  Taxa_Erro
0  RoBERTa  0.955377  0.956221  0.955377  0.955667  0.88445   0.044623
```

Writing model shards: 100% 1/1 [00:12<00:00, 12.64s/it]

Writing model shards: 100% 1/1 [00:12<00:00, 12.20s/it]

4. Predição de Novo Comentário

```
### --- NOVOS COMENTÁRIOS: --- ###

uploaded = files.upload()

novos_comentarios = pd.read_csv(io.BytesIO(uploaded['Comentarios.csv']), sep=';')
```

Escolher Ficheiros Comentarios.csv

Comentarios.csv(text/csv) - 255 bytes, last modified: 12/05/2026 - 100% done

Saving Comentarios.csv to Comentarios.csv

Writing model shards: 100% 1/1 [00:47<00:00, 47.06s/it]

Writing model shards: 100% 1/1 [00:32<00:00, 32.58s/it]

Writing model shards: 100% 1/1 [00:33<00:00, 33.73s/it]

Writing model shards: 100% 1/1 [00:38<00:00, 38.68s/it]

Writing model shards: 100% 1/1 [00:46<00:00, 46.37s/it]

4.1. Pré-processamento

```
# Remover reviews com sentimento neutro
novos_comentarios = novos_comentarios[novos_comentarios['Sentiment'] != 'Neutral']

# Visualizar as primeiras linhas
display(novos_comentarios.head(10))
```

Writing model shards: 100% 1/1 [00:44<00:00, 44.80s/it]

	Comentarios_Novos	Sentiment
0	This app is a true paragon.	Positive
1	The interface remains totally execrable.	Negative
3	Simply effulgent and highly efficient!	Positive
4	Navigation is inimical to usage.	Negative

Writing model shards: 100% 1/1 [00:20<00:00, 20.52s/it]
Writing model shards: 100% 1/1 [00:22<00:00, 22.05s/it]
Writing model shards: 100% 1/1 [00:40<00:00, 40.88s/it]
Writing model shards: 100% 1/1 [00:25<00:00, 25.95s/it]
Writing model shards: 100% 1/1 [00:35<00:00, 35.80s/it]
Writing model shards: 100% 1/1 [00:44<00:00, 44.67s/it]
Writing model shards: 100% 1/1 [00:17<00:00, 17.52s/it]

4.2. Avaliação de novos comentários

```
# Extrair textos
textos = novos_comentarios["Comentarios_Novos"].astype(str).tolist()

# Tokenizar em batch
tokens = tokenizer(
    textos,
    truncation=True,
    padding=True,
    max_length=128,
    return_tensors="pt"
)

# Garantir que o modelo está na GPU (se existir)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Enviar tokens para o mesmo device
tokens = {k: v.to(device) for k, v in tokens.items()}

# Colocar modelo em modo avaliação
model.eval()

# Inferência
with torch.no_grad():
    outputs = model(**tokens)

logits = outputs.logits

# Previsões
classe_prevista = np.argmax(logits.cpu().numpy(), axis=1)

# Converter para labels originais
sentimento = le.inverse_transform(classe_prevista)

# Guardar no dataframe
novos_comentarios["Sentimento_previsto"] = sentimento
```